

Document Number:	P0171R0
Date:	2015-11-06
Audience:	Evolution
Revises:	none
Reply to:	Gor Nishanov (gorn@microsoft.com)

Response To: Resumable Expressions P0114R0

This paper is in response to concerns expressed in the resumable expressions paper [P0114](#). We include brief quotes as a quick reminder of the concerns being discussed. Readers are encouraged to refer to [P0114](#) for full context. Familiarity with [P0054](#), [P0055](#), [P0057](#) and [N4402](#) is highly desired.

When scheduling logic embedded in the language

(Section 1.1) ... design choices in N4402 ... introduce the potential for unfairness and starvation.
(Section 4.5) This design, where `await` prefers to avoid coroutine suspension if it can, is a bad default. It leads to unfairness, jitter, and starvation.

We completely agree with [P0114](#) that

- scheduling logic should not be embedded in the language
- favoring synchronous completion leads to unfairness, jitter, and starvation.

Luckily [P0057](#) does none of those things. [P0057](#) offers pure syntactic sugar and the responsibility for making such decisions is left to the library writer. In this particular case, it is not a job of a compiler to second guess a library-provided awaitable that says that `suspend` is not required.

In this concrete case, the responsibility lies with the implementor of

```
future<Message> receive_message(Connection& c);
```

In a case of a synchronous completion, a good async I/O library writer would have returned a future that is not ready and provided the result in `.get()` or to `.then`. In fact, to even make such a future as described in [P0114](#), the library writer had to do the work to avoid safe defaults selected by the operating system.

For example, on Windows, even for synchronous API completions, notifications are always sent to a completion queue to be processed in the same way as any asynchronous completion. To suppress this behavior, a library writer would have to write the following:

```
// ascertain that a particular socket provider has support for sync completions:

WSAPROTOCOL_INFOW wsaProtocolInfo;
int wsaProtocolInfoSize = sizeof(wsaProtocolInfo);
iResult = getsockopt(
    s.native_handle(),
    SOL_SOCKET,
    SO_PROTOCOL_INFOW,
    reinterpret_cast<char*>(&wsaProtocolInfo),
    &wsaProtocolInfoSize);
panic_if(iResult == SOCKET_ERROR, "SOL_SOCKET");

syncCompletionPossible = (wsaProtocolInfo.dwServiceFlags1 & XP1_IFS_HANDLES) != 0;

if (syncCompletionPossible)
    SetFileCompletionNotificationModes(s.native_handle(),
        FILE_SKIP_COMPLETION_PORT_ON_SUCCESS);
```

Even in those cases where library writer have chosen suboptimal behavior for awaitables returned from their asynchronous APIs, such as `receive_message` above, [P0054](#) offers `await_transform` customization point which allows coroutine author to override defaults provided by the library writer.

We must remember to annotate the call with an `await` keyword

(Section 4.1): ... whenever we make a call to a resumable function we must remember to annotate the call with an `await` keyword. Failure to do so can result in hard to diagnose bugs ...

We completely agree that as the C++ language stands today, it is too easy to forget to handle the result of the function / constructor. We can see those mistakes made with terrifying frequency. It is so bad that some companies resorted to using macros, such as `ACQUIRE_LOCK(m)` as to avoid making the mistake of these kind:

```
lock_guard<mutex> (m); // oops, lock is acquired and released before ';'
    instead of
lock_guard<mutex> _(m); // aha, now it will work as we wanted
```

Similarly, one of the famous misuses of `std::async` results in the following code executing a, b, c and d sequentially.

```
a();
std::async([]{b();});
std::async([]{c();});
d();
```

We were working on a proposal to address that, but, luckily, Andrew Tomazos beat us to it and offered [NoDiscard](#) paper addressing this concern. Annotating functions/classes with `[[nodiscard]]` attribute allows to catch mistakes at compile time as shown in the example below.

```
[[nodiscard]] future<int> Connection::Read(void* buf, size_t len);

template <typename Mutex> [[nodiscard]] class lock_guard;

future<void> f(Connection& c) {
    ...
    c.Read(buf, sizeof(buf)); // error: the return value of Read is discarded
    lock_guard<mutex> (m); // error: the return value of the constructor is discarded
}
```

[If we had] direct access to the implementation type `__foo`, we could write an efficient ... batching_generator

All generator implementations shown in papers [N4402](#) (and previous versions), [P0057](#) (and previous versions) rely on direct access to the relevant parts of the coroutine data structures.

Let's take a look at [N4402](#) "appendix A" that shows an example of a generator. We pasted here relevant snippets:

```
template <typename T>
struct generator {
    struct promise_type {
        T const * CurrentValue;
        ...
        void yield_value(T const& value) { CurrentValue = addressof(value); }
    };
    struct iterator : std::iterator<input_iterator_tag, T> {
        coroutine_handle<promise_type> coro;
        T const& operator*() const { return *coro.promise().CurrentValue; }
        ...
    };
};
```

An object of a `promise_type` type resides inside of the coroutine internal state and direct access is provided by `coroutine_handle::promise()`.

In this example, when control flow reaches `yield expr;`, the coroutine is suspended, a pointer to an expression is stored in the coroutine promise and the consumer dereferences the iterator and gets direct access to the expression being yielded without any moves or copies.

It is trivial to write a `batching_generator<T>`, but in this case, a simple generator as shown in [N4402](#) or [P0057](#) will be more efficient. Because...

What compiler giveth compiler taketh away

```
(Section 3) ... we are not required to use a type-erased container
(Section 4.3) Type erasure and fine grained polymorphism
    Every value produced by the generator requires an indirect function call.
    By embedding fine-grained type erasure into the language, we are denied
    the opportunity to develop library abstractions that amortise the cost of
    runtime polymorphism.
(Section 4.4) Rather than minimising the abstraction penalty, a consequence of
    N4402 resumable functions' type erasure is that they exhibit poor scalability
```

These indeed are grave concerns but they are inapplicable to [P0057](#). As discussed in the Urbana, [P0057](#) (and Urbana's [N4134](#)) allows a compiler to choose

under which conditions type erasure needs to be performed and when it can be elided.

When a coroutine's lifetime is fully enclosed in the lifetime of its caller, no type erasure, indirect calls or memory allocations need to be done, which are exactly the cases when [P0114](#) does not need type erasure. However, when you want to put a coroutine on an ABI boundary and/or use it for asynchronous scenarios, the compiler will do it for you which avoids the need to write boilerplate code. Moreover, in Urbana we discussed how this optimization can work even across ABI boundaries.

We are not aware of any circumstances where a solution written using the abstractions of [P0114](#) will be more efficient than the same problem being solved using abstractions offered by [P0057](#) and we have a case where abstractions of [P0057](#) results in strictly better efficiency than [P0114](#). For example, compare the negative overhead await adapters from [N4287](#) with await library emulation shown in [P0114](#).

A very costly maintenance nightmare

```
(Section 3) ... the maintenance burden of a viral await keyword
(Section 4.1) Experiences with other languages that take a similar approach
(such as Python) have shown that this quickly becomes a very costly maintenance nightmare.
```

We weren't able to obtain the source for the information for the claims above, so we had to resort to our own research.

Since C# was the first language that acquired this exact syntax in 2012, we contacted the C# design team and the C# program management teams to learn about their experience. They were not aware of any studies on that matter and commented that delight and joy is the usual reaction of people who use the `await` feature extensively.

Moreover, the await syntax proved extremely useful and popular and more languages adopted it to solve their asynchronous blues, [HACK](#) and [Dart](#) in 2014, Python ([PEP0492](#)) in May of 2015 and it seems to be on track to get into [EcmaScript7](#) soon.

Since we are on the subject of maintenance nightmares, we would like to offer a conjecture that the absence of the await in [P0114](#) is a likely source of many maintenance nightmares. Without a syntactic marker to signal to the person reading the code that something funny is going on, it is impossible to tell whether the following code is correct or not:

```
auto foo(string & s) {
    s.push_back('<');
    do-stuff
    s.push_back('>');
}
```

How can we be sure that *do-stuff* will never result in suspension? Should we follow all of the possible call sequences to make sure that none of them lead to `break resumable;?`

Note, that this code is fine to be used with fibers or threads, because in that case the entire stack is suspended, so the caller which owns `s` will be suspended as well, thus avoiding mismatched lifetimes.

In coroutines the suspend point is clearly marked with `await`, which tells the reader that something unusual happens in this function and allows the reader, for example, to confirm whether the lifetimes of the objects of interest align with the lifetime of the coroutine or not, whether some locks need to be acquired to protect some concurrently accessed data, and whether some locks need to be released before the execution reaches the suspend point.

Note, that [P0054](#) describes how such 'no-await-required' behavior can be optionally built on top of [P0057](#), however, due to our grave concerns about maintainability of such code we recommend against it.

Island of abstractions

```
(Section 4.2) Over time we may end up with a complete set of await-enabled algorithms
that mirror the existing standard algorithms. These algorithms will be
independent and not interoperable with the existing algorithms
```

Does it have to be the way [P0114](#) describes? We don't believe so.

Embracing coroutines makes existing libraries better and leads to smaller API surface.

Paper [P0055](#) shows how to extend the CompletionToken model so that any library that follows that model will automatically gain “negative-overhead” behavior compared to what they get with a callback model and retain the performance if they use callbacks.

The beauty of it is that it is automatic. Providing a low level API with a particular shape and a trivial templated high level wrapper allows the CompletionToken transform to do the magic. When used with coroutines, suddenly the libraries that used to do memory allocation and type erasure for every single async operation, no longer will have to. This affects the networking proposal, boost::asio, executors, etc. An additional benefit is that it reduces the number of APIs that a library needs to provide as shown at the end of [P0055](#) paper.

The benefit of this approach extends beyond the networking library to other future standard or non-standard libraries modeling their APIs on the

Conclusion

We agree with most of the concerns expressed by [P0114](#), however all but one are not relevant to [P0057](#). The concern that is relevant (mistake of not using the result of a function/constructor) is not unique to coroutines and is tackled by paper [NoDiscard](#).

Our position is that for any concrete problem that can be solved with the abstractions offered by [P0114](#), the same problem can be solved with the abstractions offered by [P0057](#) with equal or superior characteristics of:

- how much code end-user have to write to solve the problem
- how much library support required to enable the abstraction
- what is the abstraction penalty

References

[N4287: Threads, Fibers and Couroutines \(slides deck\)](#)

[N4402: Resumable Functions \(revision 4\)](#)

[P0054: Coroutines: Reports from the field](#)

[P0055: On Interactions Between Coroutines and Networking Library](#)

[P0057: Wording for Coroutines rev3](#)

[p0114: Resumable Expressions \(revision 1\)](#)

[NoDiscard: P0068: Proposal of "unused", "nodiscard" and "fallthrough" attributes.](#)

[PEP0492: Coroutines with async and await syntax](#)

[EcmaScript7: JavaScript goes to Asynchronous city](#)

[HACK: Hack Language Reference](#)

[Dart: Spicing Up Dart with Side Effects](#)